

rtracklayer - Data Import

Kasper D. Hansen

- [Dependencies](#)
- [Corrections](#)
- [Overview](#)
- [Other Resources](#)
- [The import function](#)
- [BED files](#)
- [BigWig files](#)
- [Other file formats](#)
- [Extensive example](#)
- [LiftOver](#)
- [Importing directly from UCSC](#)
- [Tabix indexing](#)
- [SessionInfo](#)

Dependencies

This document has the following dependencies:

```
library(rtracklayer)
library(AnnotationHub)
library(Rsamtools)
```

Use the following commands to install these packages in R.

```
source("http://www.bioconductor.org/biocLite.R")
biocLite(c("rtracklayer", "AnnotationHub", "Rsamtools"))
```

Corrections

Improvements and corrections to this document can be submitted on its [GitHub](#) in its [repository](#).

Overview

The *rtracklayer* package interfaces to (UCSC) Genome Browser. It contains functions for importing and exporting data to this browser.

This includes functionality for parsing file formats associated the UCSC Genome Browser such as BED, Wig, BigBed and BigWig.

Other Resources

- The vignette from the [rtracklayer webpage](#).

The import function

The function to parse data formats is `import()`. This function has a `format` argument taking values such as BED or Bigwig.

Note that there is a help page for the general `import()` function, but there are also file format specific help pages. The easiest way to get to these help pages is to look for `xxFile` with `xx` being the format.

```
?import  
?BigwigFile
```

There are often format specific arguments.

BED files

Most BED files are small and can be read as a single object. The output of `import(format = "BED")` is a `GRanges`.

You can specify genome (for example hg19) and the function will try to make an effort to populated the `seqinfo` of the `GRanges`.

You can also use the `which` argument to selectively parse a subset of the file which overlaps a `GRanges`. This becomes much more efficient if the file has been tabix-indexed (see below).

BigWig files

BigWig files typically store whole-genome coverage vectors (or at least whole-genome data). For this reason, the R representation of a BigWig file is usually quite big, so it might be necessary to read it into R in small chunks.

As for BED files, `import(format="Bigwig")` supports a `which` argument which is a `GRanges`. Its output data type is a `GRanges` per default, but using the `as` argument you can have `as="Rle"` and a few other options.

The `import(format="Bigwig")` does not support a `genome` argument.

Other file formats

- GFF
- TwoBit
- Wig
- bedGraph

Extensive example

Let us start an `AnnotationHub`:

```
library(AnnotationHub)
ahub <- AnnotationHub()
table(ahub$rdataclass)
```

```
##
##      BigwigFile      biopax      ChainFile      data.frame
##           10143           9           1113           6
##      ExpressionSet      FaFile      GRanges      Inparanoid8Db
##           1           5112           17365           268
##           OrgDb SQLiteConnection      TwoBitFile      vcfFile
##           1164           1           144           38
```

At this point, you should have seen several of these file formats mentioned. The `GRanges` are usually directly constructed from BED file, and the `seqinfo` information is fully populated:

```
ahub.gr <- subset(ahub, rdataclass == "GRanges" & species == "Homo sapie  
gr <- ahub.gr[[1]]  
gr
```



GRanges object with 83523 ranges and 5 metadata columns:

```
##           seqnames           ranges strand |
##           <Rle>           <IRanges> <Rle> |
##      [1]   chr11 [ 65266509,  65266572]   + |
##      [2]    chr1 [156675257, 156675415]   - |
##      [3]   chr10 [ 33247091,  33247233]   - |
##      [4]    chrM [   10469,    10473]     + |
##      [5]   chr15 [ 69745106,  69745303]     + |
##      ...      ...      ...      ...
## [83519]   chrX [100078869, 100078951]     + |
## [83520]   chrX [110535150, 110535159]     - |
## [83521]   chrX [118371391, 118371473]     + |
## [83522]   chrX [ 64254602,  64254777]     - |
## [83523]   chrX [ 76889467,  76889479]     - |
##
##                                name      score      leve
##                                <character> <integer> <numeric>
##      [1] chr11:65266509:65266572:+:0.04:1.000000      0 20993.67
##      [2] chr1:156675257:156675415:-:0.35:1.000000      0 11580.47
##      [3] chr10:33247091:33247233:-:0.32:1.000000      0  8098.09
##      [4]      chrM:10469:10473:+:4.32:0.000017      0  7538.40
##      [5] chr15:69745106:69745303:+:1.62:1.000000      0  6673.18
##      ...      ...      ...      ...
## [83519] chrX:100078869:100078951:+:2.69:0.003504      0    0.482
## [83520] chrX:110535150:110535159:-:3.77:0.000296      0    2.389
## [83521] chrX:118371391:118371473:+:0.28:0.011726      0    0.482
## [83522] chrX:64254602:64254777:-:2.53:1.000000      0    0.580
## [83523] chrX:76889467:76889479:-:1.26:0.000001      0    0.482
##
##           signif      score2
##           <numeric> <integer>
##      [1] 3.36e-10      0
##      [2] 3.54e-10      0
##      [3] 3.82e-10      0
##      [4] 3.82e-10      0
##      [5] 4.09e-10      0
##      ...      ...      ...
## [83519]      1      0
## [83520]      1      0
## [83521]      1      0
## [83522]      1      0
## [83523]      1      0
## -----
## seqinfo: 24 sequences from hg19 genome
```

```
seqinfo(gr)
```

```
## Seqinfo object with 24 sequences from hg19 genome:
##   seqnames seqlengths isCircular genome
##   chr1      249250621      FALSE   hg19
##   chr10     135534747      FALSE   hg19
##   chr11     135006516      FALSE   hg19
##   chr12     133851895      FALSE   hg19
##   chr13     115169878      FALSE   hg19
##   ...       ...           ...     ...
##   chr7      159138663      FALSE   hg19
##   chr8      146364022      FALSE   hg19
##   chr9      141213431      FALSE   hg19
##   chrM           16571      FALSE   hg19
##   chrX      155270560      FALSE   hg19
```

Perhaps more interesting is the data in form of BigWig files.

```
ahub.bw <- subset(ahub, rdataclass == "BigwigFile" & species == "Homo sa
ahub.bw
```



```
## AnnotationHub with 9828 records
## # snapshotDate(): 2015-08-17
## # $dataProvider: BroadInstitute
## # $species: Homo sapiens
## # $rdataclass: BigwigFile
## # additional mcols(): taxonomyid, genome, description, tags,
## #   sourceurl, sourcetype
## # retrieve records with, e.g., 'object[["AH32002"]]'
##
##           title
## AH32002 | E001-H3K4me1.fc.signal.bigwig
## AH32003 | E001-H3K4me3.fc.signal.bigwig
## AH32004 | E001-H3K9ac.fc.signal.bigwig
## AH32005 | E001-H3K9me3.fc.signal.bigwig
## AH32006 | E001-H3K27me3.fc.signal.bigwig
## ...      ...
## AH41825 | E125-H4K91ac.imputed.pval.signal.bigwig
## AH41826 | E126-H4K91ac.imputed.pval.signal.bigwig
## AH41827 | E127-H4K91ac.imputed.pval.signal.bigwig
## AH41828 | E128-H4K91ac.imputed.pval.signal.bigwig
## AH41829 | E129-H4K91ac.imputed.pval.signal.bigwig
```

```
bw <- ahub.bw[[1]]
bw
```

```
## BigwigFile object
## resource: /Users/khansen/.AnnotationHub/37442
```

This returns us a file name, ready for use by import.

```
gr1 <- gr[1:3]
out.gr <- import(bw, which = gr1)
out.gr
```

```
## GRanges object with 14 ranges and 1 metadata column:
##           seqnames           ranges strand |           score
##           <Rle>           <IRanges> <Rle> |           <numeric>
## [1]      chr1 [156675257, 156675262]      * | 0.634060025215149
## [2]      chr1 [156675263, 156675285]      * | 0.501269996166229
## [3]      chr1 [156675286, 156675297]      * | 0.250600010156631
## [4]      chr1 [156675298, 156675307]      * | 0.315039992332458
## [5]      chr1 [156675308, 156675332]      * | 0.303939998149872
## ...      ...      ...      ...      ...
## [10]     chr1 [156675392, 156675397]      * | 0.303939998149872
## [11]     chr1 [156675398, 156675401]      * | 0.250600010156631
## [12]     chr1 [156675402, 156675415]      * | 0
## [13]    chr10 [ 33247091,  33247233]      * | 0
## [14]    chr11 [ 65266509,  65266572]      * | 0
## -----
## seqinfo: 25 sequences from an unspecified genome
```

This gives us the content in the form of a GRanges. Often, an Rle might be appropriate:

```
out.rle <- import(bw, which = gr1, as = "Rle")
out.rle
```



```
## RleList of length 25
## $chr1
## numeric-Rle of length 249250621 with 13 runs
##   Lengths:      156675256              6 ...      92575220
##   values :      0 0.634060025215149 ...      0
##
## $chr10
## numeric-Rle of length 135534747 with 1 run
##   Lengths: 135534747
##   values :      0
##
## $chr11
## numeric-Rle of length 135006516 with 1 run
##   Lengths: 135006516
##   values :      0
##
## $chr12
## numeric-Rle of length 133851895 with 1 run
##   Lengths: 133851895
##   values :      0
##
## $chr13
## numeric-Rle of length 115169878 with 1 run
##   Lengths: 115169878
##   values :      0
##
## ...
## <20 more elements>
```

You can get all of chr22 by

```
gr.chr22 <- GRanges(seqnames = "chr22",
                    ranges = IRanges(start = 1, end = seqlengths(gr)["chr22"]))
out.chr22 <- import(bw, which = gr.chr22, as = "Rle")
out.chr22[["chr22"]]
```

```
## numeric-Rle of length 51304566 with 1381642 runs
##   Lengths:      16050196              194 ...      61301
##   values :      0 0.465829998254776 ...      0
```

LiftOver

LiftOver is a popular tool from the UCSC Genome Browser for converting between different genome versions. The *rtracklayer* package also exposes this function through the `liftOver`. To use `liftOver` you need a so-called “chain” file describing how to convert from one genome to another. This can be obtained by hand from UCSC, or directly from [AnnotationHub](#).

We can re-use our AnnotationHub:

```
ahub.chain <- subset(ahub, rdataclass == "ChainFile" & species == "Homo  
query(ahub.chain, c("hg18", "hg19"))
```

```
## AnnotationHub with 2 records  
## # snapshotDate(): 2015-08-17  
## # $dataprovder: UCSC  
## # $species: Homo sapiens  
## # $rdataclass: ChainFile  
## # additional mcols(): taxonomyid, genome, description, tags,  
## #   sourceurl, sourcetype  
## # retrieve records with, e.g., 'object[["AH14149"]]'  
##  
##           title  
## AH14149 | hg19ToHg18.over.chain.gz  
## AH14220 | hg18ToHg19.over.chain.gz
```

```
chain <- ahub.chain[ahub.chain$title == "hg19ToHg18.over.chain.gz"]  
chain <- chain[[1]]  
gr.hg18 <- liftOver(gr, chain)  
gr.hg18
```

```
## GRangesList object of length 83523:
## $1
## GRanges object with 1 range and 5 metadata columns:
##      seqnames      ranges strand |
##      <Rle>          <IRanges>  <Rle> |
## [1] chr11 [65023085, 65023148] + |
##                                name      score      level
##                                <character> <integer> <numeric>
## [1] chr11:65266509:65266572:+:0.04:1.000000      0 20993.67
##      signif      score2
##      <numeric> <integer>
## [1] 3.36e-10      0
##
## $2
## GRanges object with 1 range and 5 metadata columns:
##      seqnames      ranges strand |
## [1] chr1 [154941881, 154942039] - |
##                                name score      level      signif
## [1] chr1:156675257:156675415:-:0.35:1.000000      0 11580.47 3.54e-1
##      score2
## [1] 0
##
## $3
## GRanges object with 1 range and 5 metadata columns:
##      seqnames      ranges strand |
## [1] chr10 [33287097, 33287239] - |
##                                name score      level      signif
## [1] chr10:33247091:33247233:-:0.32:1.000000      0 8098.093 3.82e-10
##      score2
## [1] 0
##
## ...
## <83520 more elements>
## -----
## seqinfo: 24 sequences from an unspecified genome; no seqlengths
```

This converts a GRanges into a GRangesList, why? This is because a single range (interval) may be split into multiple intervals in the other genome. So each element in the output correspond to a single range in the input. If the ranges are small, most ranges should be mapped to a single range. Let us look at the number of elements in output:

```
table(elementLengths(gr.hg18))
```

```
##
##      0      1      2      3
##    47 83468      6      2
```

Only a few ranges were not mapped and only a few were split.

Importing directly from UCSC

Using [*rtracklayer*](#) you can import tables and tracks directly from the UCSC Genome Browser. However, it is now possible to get a lot (all?) of this data from [*AnnotationHub*](#) and this later package seems friendlier.

It is possible that not all tracks / tables and/or all information from the different track / tables from UCSC are exposed in [*AnnotationHub*](#).

See a detailed exposition in the package vignette.

Tabix indexing

Tabix indexing is a way to index a text file with chromosomal positions for random access. This will greatly speed up any querying of such a file. The [*tabix*](#) functionality was introduced in the SAMtools library; this library was later renamed to htlib.

Tabix indexing is usually something you do at the command line, but there is also the convenient possibility of doing it from inside Bioconductor using [*indexTabix*](#) from the [*Rsamtools*](#) package. First however, the file needs to be bgzip2 compressed, which you can do using the `bgzip` function. A full pipeline, using an example SAM file from [*Rsamtools*](#) is

```
library(Rsamtools)
from <- system.file("extdata", "ex1.sam", package="Rsamtools",
                    mustWork=TRUE)
from
```

```
## [1] "/Library/Frameworks/R.framework/Versions/3.2/Resources/library/R
```



```
to <- tempfile()
zipped <- bgzip(from, to)
idx <- indexTabix(zipped, "sam")
```

see also the help page for `indexTabix`.

SessionInfo

```
## R version 3.2.1 (2015-06-18)
## Platform: x86_64-apple-darwin13.4.0 (64-bit)
## Running under: OS X 10.10.5 (Yosemite)
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## attached base packages:
## [1] stats4      parallel  methods    stats      graphics  grDevices  utils
## [8] datasets    base
##
## other attached packages:
##  [1] Rsamtools_1.20.4      Biostrings_2.36.3      XVector_0.8.0
##  [4] AnnotationHub_2.0.3    rtracklayer_1.28.8      GenomicRanges_1.20.5
##  [7] GenomeInfoDb_1.4.2     IRanges_2.2.7           S4Vectors_0.6.3
## [10] BiocGenerics_0.14.0    BiocStyle_1.6.0         rmarkdown_0.7
##
## loaded via a namespace (and not attached):
##  [1] Rcpp_0.12.0              BiocInstaller_1.18.4
##  [3] formatR_1.2              futile.logger_1.4.1
##  [5] bitops_1.0-6             futile.options_1.0.0
##  [7] tools_3.2.1              zlibbioc_1.14.0
##  [9] digest_0.6.8             evaluate_0.7.2
## [11] RSQLite_1.0.0            shiny_0.12.2
## [13] DBI_0.3.1                curl_0.9.2
## [15] yaml_2.1.13              stringr_1.0.0
## [17] httr_1.0.0               knitr_1.11
## [19] Biobase_2.28.0           R6_2.1.0
## [21] AnnotationDbi_1.30.1     XML_3.98-1.3
## [23] BiocParallel_1.2.20      lambda.r_1.1.7
## [25] magrittr_1.5             htmltools_0.2.6
## [27] GenomicAlignments_1.4.1  mime_0.3
## [29] interactiveDisplayBase_1.6.0 xtable_1.7-4
## [31] httpuv_1.3.3             stringi_0.5-5
## [33] RCurl_1.95-4.7
```